

Part 2 – Docker Images & Dockerfile

Instead of manually installing software every time, we describe how to build an image.

The instructions are written in a **Dockerfile**.

Project

```
|  
|— Dockerfile  
|— package.json  
|— server.js  
|— ...
```

FROM

Defines the base image.

FROM node:22

Everything starts from another image.

Examples

FROM python:3.13

FROM ubuntu

FROM nginx

WORKDIR

Sets the working directory.

WORKDIR /app

Equivalent to

cd /app

COPY

Copies files.

COPY . .

Meaning

Current Project



/app inside image

Copy only **package.json**

COPY package.json .

RUN

Runs commands **during image build**.

Example

RUN npm install

or

RUN apt update

Important:

RUN executes **once while building the image**, not every time the container starts.

CMD

Defines the default command when the container starts.

Example

CMD ["node","server.js"]

When running

docker **run** myapp

Docker automatically executes

node server.js

EXPOSE

Documents which port the application listens on.

EXPOSE 3000

This **does not publish the port**. You still need

-p 3000:3000

when running the container.

Example Dockerfile

FROM node:22

WORKDIR /app

COPY package*.json ./

RUN npm install

COPY . .

EXPOSE 3000

CMD ["node","server.js"]

Build Image

docker build -t myapp .

Explanation

-t myapp

gives the image a **name**.

.

means

Current folder

Run Your Image

docker run -p 3000:3000 myapp

Tag Versions

docker build -t myapp:v1 .

Later

docker build -t myapp:v2 .

Run

docker run myapp:v1

.dockerignore

Works like `.gitignore`.

Files listed here are **not copied into the build context**, making builds faster and images smaller.

Example

```
node_modules
.git
.env
*.log
coverage
```

Without `.dockerignore`, Docker may upload hundreds of megabytes unnecessarily during the build.

Image Lifecycle

